

C - The If Statement

Conditional execution of instructions is the basic requirement of a computer program. The **if** statement in C is the primary conditional statement. C allows an optional **else** keyword to specify the statements to be executed if the **if** condition is false.

C - if Statement

The **if** statement is a fundamental **decision control statement** in C programming. One or more statements in a block will get executed depending on whether the Boolean condition in the **if** statement is true or false.

Syntax of if Statement

The **if** statement is written with the following syntax –

```
if(boolean_expression) {  
    /* statement(s) will execute if the boolean expression is true */  
}
```

Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

How if Statement Works?

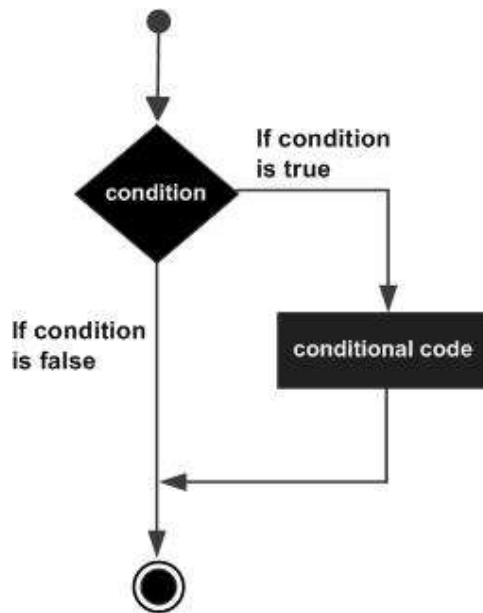
C uses a pair of curly brackets to form a code block. If the Boolean expression evaluates to true, then the block of code inside the **if** statement will be executed.

If the Boolean expression evaluates to false, then the first set of code after the end of the **if** statement (after the closing curly brace) will be executed.

C programming treats any non-zero and non-null values as true. And if the values are either zero or null, then they are treated as false values.

Flowchart of if Statement

The behaviour of the **if** statement is expressed by the following flowchart –



Flowchart Explained

When the program control comes across the **if** statement, the condition is evaluated.

If the condition is true, the statements inside the **if** block are executed.

If the condition is false, the program flow bypasses the conditional block.

Statements after the **if** block are executed to continue the program flow.

Example of if Statement in C

This example demonstrates the simplest use-case of **if** statement. It determines and tells the user if the value of a variable is less than 20.

</>

Open Compiler

```
#include <stdio.h>

int main (){

    /* local variable declaration */
    int a;

    // run the program for different values of "a"
    // Assign 12 first and 40 afterwards

    a = 12; //change to 40 and run again
    printf("Value of a is : %d\n", a);
```

```
// check the boolean condition using if statement

if(a < 20){
    //if the condition is true, then print the following
    printf("a is less than 20\n" );
}
return 0;
}
```

Output

Run the above program and check its output –

```
Value of a is : 12
a is less than 20
```

Now assign a number greater than 20. The **if** condition is not executed.

```
Value of a is: 40
```

if Statement with Logical Operations

You can put a compound boolean expression with the use of **&&** or **||** operators in the parenthesis in the **if** statement.

Example

In the following example, three **variables** "a", "b" and "c" are compared. The **if** block will be executed when "a" is greater than both "b" and "c".

</>

Open Compiler

```
#include <stdio.h>

int main () {

    /* local variable declaration */
    int a, b, c;
```

```

/*use different values for a, b and c as
10, 5, 7
10, 20, 15
*/

// change to 10,20,15 respectively next time
a = 10; b = 5; c = 7;

if (a>=b && a>=c){
    printf ("a is greater than b and c \n");
}
printf("a: %d b:%d c:%d", a, b, c);

return 0;
}

```

Output

Run the code and check its output –

```

//when values for a, b and c are 10 5 7
a is greater than b and c
a: 10 b:5 c:7

//when values for a, b and c are 10 20 15
a: 10 b:20 c:15

```

Note that the statement following the conditional block is executed after the block is executed. If the condition is false, the program jumps directly to the statement after the block.

Multiple if Statements

If you have multiple conditions to check, you can use the if statement multiple times.

Example

In this example, the net payable amount is calculated by applying discount on the bill amount.

The discount applicable is 5 percent if the amount is between 1000 to 5000, and 10 percent if the amount is above 5000. No discount is applicable for purchases below 1000.

[Open Compiler](#)

```
#include <stdio.h>

int main () {

    // local variable declaration
    int amount;
    float discount, net;

    /*Run the program for different values
    of amount - 500, 2250 and 5200. Blocks in
    respective conditions will be executed*/

    // change to 2250 and 5200 and run again
    amount = 500;

    if (amount < 1000){
        discount=0;
    }
    if (amount >= 1000 && amount<5000){
        discount=5;
    }
    if (amount >= 5000){
        discount=10;
    }
    net = amount - amount*discount/100;
    printf("Amount: %d Discount: %f Net payable: %f", amount, discount, net);

    return 0;
}
```

Output

```
//when the bill amount is 500
Amount: 500 Discount: 0.000000 Net payable: 500.000000
```

```
//when the bill amount is 2250
Amount: 2250 Discount: 5.000000 Net payable: 2137.500000
```

```
//when the bill amount is 5200
```

```
Amount: 5200 Discount: 10.000000 Net payable: 4680.000000
```

Checking Multiple Conditions With if Statement

You can also check multiple conditions using the logical operators in a single if statement.

Example

In this program, a student is declared as **passed** only if the average of "phy" and "maths" marks is greater than equal to 50. Also, the student should have secured more than 35 marks in both the subjects. Otherwise, the student is declared as **failed**.

[Open Compiler](#)

```
#include <stdio.h>

int main (){
    /* local variable declaration */
    int phy, maths;
    float avg;

    /*use different values of phy and maths
    to check conditional execution*/

    //change to 40, 40 and 80, 40
    phy = 50; maths = 50;
    avg = (float)(phy + maths)/2;
    printf("Phy: %d Maths: %d Avg: %f\n", phy, maths, avg);

    if (avg >= 50 && (maths >= 35 && phy >= 35)){
        printf("Result: Pass");
    }

    if (avg<50) {
        printf("Result: Fail\n");
    }
    return 0;
}
```

Output

Run the code and check its output –

```
//when marks in Phy and Maths - 50 50  
Phy: 50 Maths: 50 Avg: 50.000000  
Result: Pass
```

```
//when marks in Phy and Maths - 40 40  
Phy: 40 Maths: 40 Avg: 40.000000  
Result: Fail
```

```
//when marks in Phy and Maths - 80 40  
Phy: 80 Maths: 40 Avg: 60.000000  
Result: Pass
```

C - The if-else Statement

The **if-else** statement is one of the frequently used **decision-making statements** in C. The **if-else** statement offers an alternative path when the condition isn't met.

The **else** keyword helps you to provide an alternative course of action to be taken when the Boolean expression in the **if** statement turns out to be false. The use of **else** keyword is optional; it's up to you whether you want to use it or not.

Syntax of if-else Statement

Here is the syntax of **if-else** clause –

```
if (Boolean expr){  
    Expression;  
    . . .  
}  
else{  
    Expression;  
    . . .  
}
```

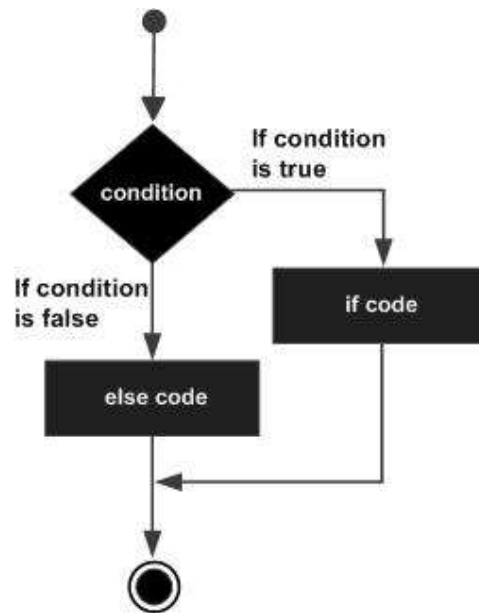
The **C compiler** evaluates the condition, and executes a statement or a block of statements following the **if statement** if it is true.

If the programming logic needs the computer to execute some other instructions when the condition is false, they are put as a part of the **else** clause.

An **if** statement is followed by an optional **else** statement, which executes when the Boolean expression is false.

Flowchart of if-else Statement

The following flowchart represents how the **if-else** clause works in C –



Note that the curly brackets in the **if** as well as the **else** clause are necessary if you have more than one statements to be executed. For example, in the following code, we don't need curly brackets.

```
if (marks<50)
    printf("Result: Fail\n");
else
    printf("Result: Pass\n");
```

However, when there are more than one statements, either in the **if** or in the **else** part, you need to tell the compiler that they need to be treated as a compound statement.

Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

C if-else Statement Examples

Example: Tax Calculation Using if-else Statement

In the code given below, the tax on employee's income is computed. If the income is below 10000, the tax is applicable at 10%. For the income above 10000, the excess income is charged at 15%.

</>

Open Compiler

```
#include <stdio.h>

int main() {
```

```

int income = 5000;
float tax;
printf("Income: %d\n", income);

if (income<10000){
    tax = (float)(income * 10 / 100);
    printf("tax: %f \n", tax);
}
else {
    tax= (float)(1000 + (income-10000) * 15 / 100);
    printf("tax: %f", tax);
}
}

```

Output

Run the code and check its output –

```

Income: 5000
tax: 500.000000

```

Set the income variable to 15000, and run the program again.

```

Income: 15000
tax: 1750.000000

```

Example: Checking Digit Using if-else Statement

The following program checks if a char variable stores a digit or a non-digit character.

</>

Open Compiler

```

#include <stdio.h>

int main() {
    char ch='7';

    if (ch>=48 && ch<=57){
        printf("The character is a digit.");
    }
}

```

```
else{
    printf("The character is not a digit.");
}
return 0;
}
```

Output

Run the code and check its output –

The character is a digit.

Assign any other character such as "*" to "ch" and see the result.

The character is not a digit.

Example: if-else Statement Without Curly Braces

Consider the following code. It intends to calculate the discount at 10% if the amount is greater than 100, and no discount otherwise.

</>

Open Compiler

```
#include <stdio.h>

int main() {
    int amount = 50;
    float discount;
    printf("Amount: %d\n", amount);

    if (amount >= 100)
        discount = amount * 10 / 100;
    printf("Discount: %f \n", discount);
    else
        printf("Discount not applicable\n");

    return 0;
}
```

Output

The program shows the following errors during the compilation –

```
error: 'else' without a previous 'if'
```

The compiler will execute the first statement after the **if** clause and assumes that since the next statement is not **else** (it is optional anyway), the subsequent `printf()` statement is unconditional. However, the next **else** is not connected to any **if** statement, hence the error.

Example: if-else Statement Without Curly Braces

Consider the following code too –

[Open Compiler](#)

```
#include <stdio.h>

int main() {
    int amount = 50;
    float discount, nett;
    printf("Amount: %d\n", amount);

    if (amount<100)
        printf("Discount not applicable\n");
    else
        printf("Discount applicable");
        discount = amount*10/100;
        nett = amount - discount;
        printf("Discount: %f Net payable: %f", discount, nett);

    return 0;
}
```

Output

The code doesn't give any compiler error, but gives incorrect output –

Amount: 50
Discount not applicable
Discount: 5.000000 Net payable: 45.000000

It produces an incorrect output because the compiler assumes that there is only one statement in the **else** clause, and the rest of the statements are unconditional.

The above two code examples emphasize the fact that when there are more than one statements in the **if** or **else** else, they must be put in curly brackets.

To be safe, it is always better to use curly brackets even for a single statement. In fact, it improves the readability of the code.

The correct solution for the above problem is shown below –

```
if (amount >= 100){  
    discount = amount * 10 / 100;  
    printf("Discount: %f \n", discount);  
} else {  
    printf("Discount not applicable\n");  
}
```

The else-if Statement in C

C also allows you to use **else-if** in the programs. Let's see where you may have to use an **else-if** clause.

Let's suppose you have a situation like this. If a condition is true, run the given block that follows. If it isn't, run the next block instead. However, if none of the above is true and all else fails, finally run another block. In such cases, you would use an **else-if** clause.

Syntax of else-if Statement

Here is the syntax of the **else-if** clause –

```
if (condition){  
    // if the condition is true,  
    // then run this code  
} else if(another_condition){  
    // if the above condition was false  
    // and this condition is true,  
    // then run the code in this block
```

```
} else{  
    // if both the above conditions are false,  
    // then run this code  
}
```

Example of else-if Statement

Take a look at the following example –

</>

Open Compiler

```
#include <stdio.h>  
  
int main(void) {  
    int age = 15;  
  
    if (age < 18) {  
        printf("You need to be over 18 years old to continue\n");  
    }else if (age < 21) {  
        printf("You need to be over 21\n");  
    } else {  
        printf("You are over 18 and older than 21 so you can continue \n");  
    }  
}
```

Output

Run the code and check its output –

You need to be over 18 years old to continue

Now, supply a different value for the variable "age" and run the code again. You will get a different output if the supplied value is less than 18.